DiamondKG - College Baseball Knowledge Graph (Neo4j)

Final Project Report

Student: Surya Narayan Swamy

Project Title: DiamondKG - College Baseball Knowledge Graph
Report Date: December 16 2025

Abstract

DiamondKG is a Neo4j-based knowledge graph built from official U.S. college baseball roster and staff pages. The project unifies player and staff information across schools into a consistent, queryable property graph that supports cross-program analysis. Over Weeks 1–7, the work progressed from a proposal and minimal ontology to a complete ETL pipeline (discovery → extraction → normalization → entity resolution → load → quality checks) and a working knowledge graph covering 20 universities for the 2025 season, with a multi-season extension strategy (2024 + 2025) enabled through reification.

The final system includes robust data cleaning for height/weight formats, explicit modeling of player attributes such as jersey, position, height in inches, weight, and BMI, as well as academic and recruiting context such as major and previous school when available. Provenance is stored for auditability using sourceUrl and retrievedAt. To support longitudinal questions (returners, departures, role changes, transfers), the project introduces a reified PlayerSeason node that stores season-specific snapshots without duplicating player identities or overwriting past data.

Table of Contents

1. Introduction
2. Project Objectives and Scope
3. Data Sources, Ethics, and Compliance
4. ETL System Architecture (Python → CSV → Neo4j)
5. Final Schema Design (Nodes + Relationships)
6. Multi-Season Modeling and Reification (PlayerSeason)
7. Weekly Progress (Weeks 1–7)
8. Data Cleaning and Normalization
9. Knowledge Graph Loading Strategy and No-Repeatability
10. Query Evaluation and Results
11. Limitations, Risks, and Mitigations

12. Changes Implemented After Feedback
13. Conclusion

1. Introduction

College baseball roster and staff information is widely available on official athletics websites, but the information is fragmented and inconsistent across schools. Differences in HTML structure (tables vs cards), formatting conventions (height/position abbreviations), and optional fields (majors, previous schools) make it difficult to compare rosters across institutions using traditional spreadsheets.

DiamondKG solves this problem by building a knowledge graph (KG) in Neo4j. A KG is well-suited for this domain because it can represent relationships such as players belonging to schools, players having positions and physical attributes, and staff belonging to programs. It also supports flexible queries and can grow incrementally as new schools, seasons, or attributes are added.

2. Project Objectives and Scope

Universities Covered

The DiamondKG knowledge graph currently includes **20 universities** (2025 roster coverage), mainly from the Northeast / Mid-Atlantic region:

- Ball State University Cardinals
- Boston College Eagles
- Brown University Bears
- Bucknell University Bison
- Butler University Bulldogs
- Cornell University Big Red
- Elon University Phoenix
- Fordham University Rams
- Gannon University Golden Knights
- Georgetown University Hoyas
- Hofstra University Pride
- Iona University Gaels
- Monmouth University Hawks
- North Carolina State University Wolfpack
- Princeton University Tigers
- Rutgers University Scarlet Knights
- Saint Joseph’s University Hawks
- University of Georgia Bulldogs

- University of Pennsylvania Quakers
- Villanova University Wildcats

GitHub Gist as the Data Publishing Layer (Python → Neo4j)

GitHub Gist provides a simple way to host small datasets and exposes each file through a stable **Raw URL**. This Raw URL is crucial because Neo4j AuraDB can import CSV files directly from an HTTPS endpoint.

Raw URLs and Neo4j import

GitHub provides a "Raw" view of each uploaded file. The Raw link becomes a direct HTTPS address that Neo4j can read during import. This avoids manual file uploads inside Neo4j and makes the workflow much more robust for repeated runs.

This approach also supports the "no-repeatability" requirement: the CSV can be re-imported multiple times while Neo4j merges into existing nodes (using deterministic IDs/constraints), preventing duplicates while still allowing updates.

Players(2024).csv -
https://gist.githubusercontent.com/suryaanarayanswamy/98c69ef01ec7416e245ed548727b3a68/raw/32118067e115276d8957de660361132a04f32467/player_2024.csv

AllCoaches.csv -
https://gist.githubusercontent.com/suryaanarayanswamy/7e627f174dca63d8022041bc4c6d9f9c/raw/ae57f9b5fd706ea3e69211f81f06d73e409b7dc1/coachess.csv

Players(2025).csv -
https://gist.githubusercontent.com/suryaanarayanswamy/15221092033f6d38ef90bb8da2ab9484/raw/295d607fe45fc329739c58dfb89773701b935bef/playerss.csv

2.1 Objectives (Weeks 1–4)

- Define a stable data model and minimal ontology.
- Start with at least 5 institutions and scale toward 20+ by the end of term.
- Build an end-to-end ETL pipeline:
  discovery → extraction → normalization → entity resolution → load.
- Track weekly progress using coverage, quality, and stability metrics.
- Coordinate overlaps with classmates for cross-checks and validation.

2.2 Scaling Objectives (Weeks 5–7)

- Expand from MVP to 15 schools (Week 5) and 20 schools (Week 7).

- Move from flat properties toward attribute modeling (nodes for jersey/height/weight/BMI).
- Improve parser robustness to handle website variability.
- Extend the model to multi-season analysis (2024 + 2025).
- Add season-aware design using reification (PlayerSeason) to enable longitudinal queries.

2.3 Scope boundaries

- Use only publicly available information from official athletics pages.
- Focus on baseball rosters and staff listings (not game results or statistics in this phase).
- Store image URLs as links (no bulk image downloads).
- Keep the graph compact and queryable while remaining extensible.

3. Data Sources, Ethics, and Compliance

3.1 Data Sources

Data is extracted from official roster and staff pages for each university's baseball program. Each record stores provenance:

- **sourceUrl**: where the record was extracted from
- **retrievedAt**: when the page was retrieved

This supports transparency, debugging, and reproducibility.

3.2 Ethical and compliance considerations

- Respect robots.txt and site terms where applicable.
- Use request delays and a respectful user agent.
- Avoid aggressive crawling.
- Limit storage to public facts, primarily text.
- Maintain provenance for auditing and corrections.

4. ETL System Architecture (Python → CSV → Neo4j)

DiamondKG uses Python for extraction and normalization, then loads cleaned data into Neo4j.

4.1 Discovery

For each school, the pipeline identifies:

- baseball roster page (players)
- coaches/staff page (staff)

4.2 Extraction (Python)

The extraction pipeline uses Python to download and parse HTML pages. Most sites are static HTML tables or card layouts. A small number may require extra handling if content is dynamically rendered.

Key design choice: a shared core scraper + per-school adjustments.
This is necessary because roster HTML differs significantly across universities.

4.3 Parsing

The parser extracts player/staff fields into structured rows. Typical player fields:

- name, jersey, position
- height, weight
- bats/throws
- hometown
- previous school (when available)
- major (when available)
- school, season
- sourceUrl, retrievedAt

Typical staff fields:

- name, title/role
- staff grouping category (when available/derived)
- school
- sourceUrl, retrievedAt

4.4 Normalization and cleaning

Major normalization tasks include:

- Height normalization into consistent feet-inches and numeric inches
- Weight cleaning into numeric form when possible
- Position cleaning (split combined positions; map to canonical categories)
- Text cleanup (trim whitespace, normalize punctuation, remove non-ASCII quotes)
- Missing-value handling (skip relationships when data is unavailable)

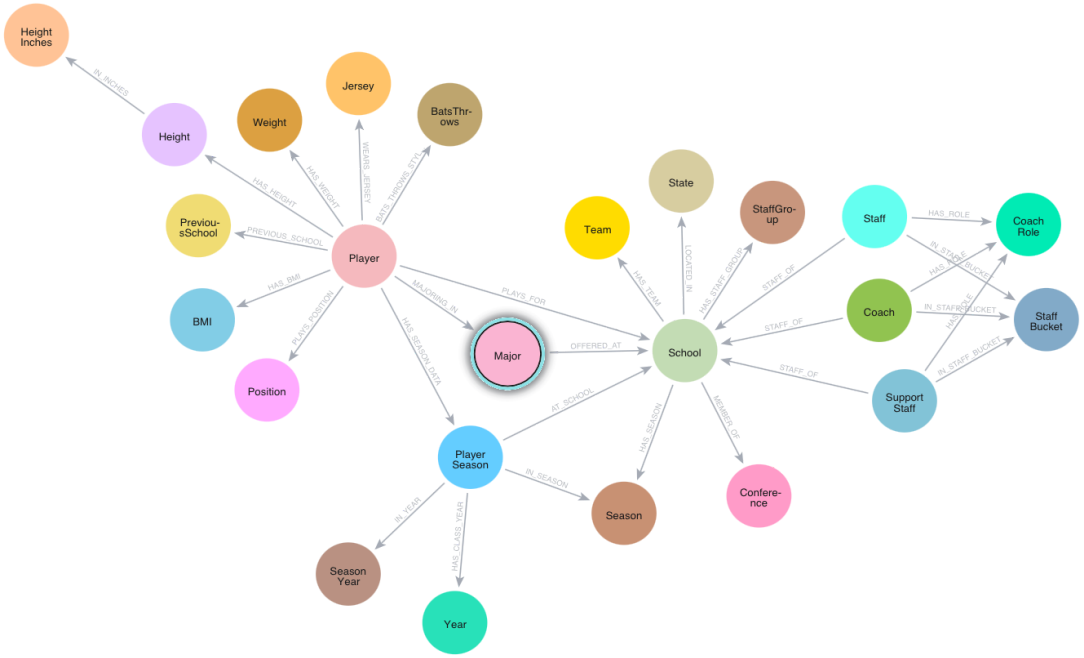
4.5 Output staging and publication

Clean outputs are exported to CSV and published to stable raw links (GitHub Gist). Neo4j imports from these raw URLs.

5. Final Schema Design (Nodes + Relationships)

This section describes the final schema as reflected in the Neo4j visualization.

5.1 Schema diagram



Neo4j visualization showing core entities (Player, School, Staff) with attribute nodes (Height, Weight, BMI, Jersey, Position, Major, PreviousSchool) and multi-season modeling via PlayerSeason linked to Season/Year.

5.2 Core nodes

- **School**: central hub representing each university program.
- **Player**: stable identity for a player (not duplicated per year).
- **PlayerSeason**: reified season snapshot (player at a school in a given season).
- **Season / Year / SeasonYear**: season context for multi-year comparisons.
- **Team**: baseball team entity linked to school (useful for conference membership).
- **Conference**: higher-level grouping for aggregated comparisons.
- **State**: geographic normalization dimension for hometown/state analysis.

5.3 Player attribute nodes (modeled as entities)

DiamondKG models frequently queried attributes as nodes so they can be shared and aggregated:

- Jersey (players who wear the same jersey connect to the same Jersey node)
- Position (players connect to positions; position normalization reduces fragmentation)
- Height and HeightInches (height inches enables numeric comparisons)
- Weight (numeric comparisons)
- BMI (computed from height/weight)
- BatsThrows (R/R, L/L, etc.)
- Major (academic program when available)
- PreviousSchool (recruiting/transfer context when available)

5.4 Coaches Bucket with Staff and Support Staff

As the DiamondKG graph scaled to more universities, I observed that athletics "staff" pages rarely list only coaches. Instead, they include a mix of roles such as head/assistant coaches, recruiting, player development, athletic training, strength & conditioning, equipment, operations, video, and communications. If all of these people are stored under a single Coach label, the graph becomes semantically incorrect and queries like "number of coaches per school" get inflated.
To solve this, I introduced a **bucketed staff model** that separates personnel into three conceptual layers: **Coach**, **Staff**, and **SupportStaff**, and then groups them through a "bucket" approach.

Bucketing approach (how the graph represents personnel)

1. **Coach (coaching personnel)**
   People with coaching responsibilities, such as Head Coach, Assistant Coach, Pitching Coach, Hitting Coach, etc.
2. **Staff (program staff)**
   Operational roles that support the team program but are not coaches (e.g., operations, recruiting coordinator, director roles, analytics/video roles, equipment).
3. **SupportStaff (health/performance/support services)**
   Roles such as athletic training, strength & conditioning, sports performance, and related support positions.

How classification is done

When parsing each staff page, the pipeline uses a **title-based classification rule**:

- Titles containing coaching keywords (e.g., "Coach", "Pitching", "Hitting", "Head Coach") are classified into the **Coach** bucket.

- Titles indicating operations/admin/recruiting/video/equipment are classified into **Staff**.
- Titles indicating training/performance/medical (e.g., “Athletic Trainer”, “Strength”, “Sports Performance”) are classified into **SupportStaff**.

This classification is lightweight but effective, and the original raw title is preserved for auditability.

Graph structure and relationships

- Each person node (Coach/Staff/SupportStaff) is linked to a School via an affiliation relationship (e.g., “STAFF_OF” or “COACHES”).
- Each person is linked to a role node (e.g., CoachRole) via a role relationship (e.g., “HAS_ROLE”).
- A grouping layer can optionally exist through nodes such as StaffGroup / StaffBucket, allowing every person to belong to exactly one bucket for clean aggregation.

What this enables

This bucket design supports the following analyses:

- **True coach counts by school** (coaching depth comparisons)
- **Support staff depth** (performance/medical investment comparisons)
- **Staff composition profiles** (how “operations-heavy” vs “coach-heavy” programs are)
- **Role distribution per bucket** (e.g., number of pitching coaches across programs)

Suggested evaluation queries (for your report, no code needed)

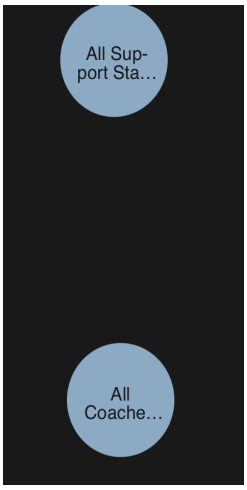
- Count **Coaches vs Staff vs SupportStaff** per school.
- Identify the **top schools by number of Coaches** (excluding non-coaching roles).
- List **most common roles** within SupportStaff (e.g., Athletic Trainer, Strength Coach).
- Compare **bucket ratios** per school (example: coaches-to-support-staff ratio).

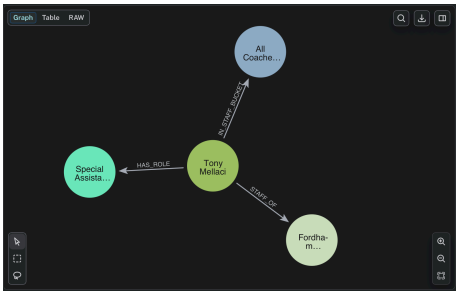
```
neo4j$ MATCH (s:School)<-[:STAFF_OF]-(p) WITH s, count(CASE WHEN p:Coach
```

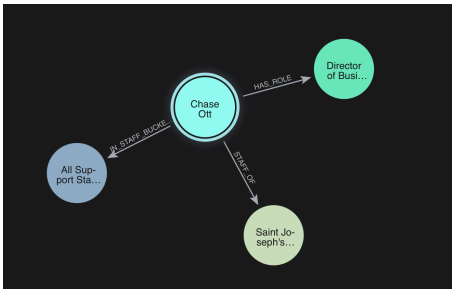
Table RAW

	School	Coaches	Staff	SupportStaff	TotalPersonnel
1	"North Carolina State University"	5	23	18	46
2	"Princeton University"	5	18	13	36
3	"Rutgers University"	4	18	14	36
4	"Fordham University"	6	13	7	26
5	"Saint Joseph's University"	5	13	8	26
6	"University of Pennsylvania"	4	13	9	26
7	"Boston College"	5	12	7	24

Started streaming 20 records after 94 ms and completed after 97 ms.







6. Multi-Season Modeling and Reification (PlayerSeason)

DiamondKG becomes significantly more powerful when moving from a single season to multiple seasons. A major design decision is the use of reification through PlayerSeason.

6.1 Why reification is necessary

Without reification, the model would connect Player directly to School and store jersey/position/weight on the Player node. That becomes incorrect as soon as 2024 and 2025 are both imported, because:

- jersey numbers can change across seasons
- positions can change
- weight and class year can change
- players can transfer schools

School	Player	Jersey_2024	Jersey_2025
"Ball State University"	"Alex Burden"	"33"	20
"Ball State University"	"Brett Griffiths"	"15"	3
"Ball State University"	"DJ Scheumann"	"7"	16
"Boston College"	"Esteban Garcia"	"51"	7

6.2 What PlayerSeason represents

PlayerSeason represents:
“This specific player at this specific school during this specific season.”

Season-scoped attributes belong to PlayerSeason:

- year/season
- jersey
- class year
- position (raw and normalized)
- weight (and possibly height if the roster changed reporting)
- major/previous school (if present for that year)
- provenance (sourceUrl, retrievedAt)

Players remain the stable identity that links PlayerSeason snapshots together.

7. Weekly Progress (Weeks 1–7)

Week 1 - Proposal

- Defined KG scope and ETL plan.
- Initial target schools: Temple, UPenn, Villanova, Saint Joseph’s, Rutgers.
- Established nodes/edges, provenance approach, and initial metrics.

Week 2 - Peer Review Alignment

- Collaborated with Chandana Ramesh and Maulesh Patel.
- Aligned core entities and naming conventions.
- Adopted deterministic IDs and mandatory provenance.
- Managed overlap rules (3 shared sites with one classmate, limited overlap overall).

Week 3 - MVP Graph (3 Schools)

- Built full pipeline for Saint Joseph's, UPenn, Villanova.
- Delivered end-to-end: scrape → clean → CSV → Neo4j load → visualization.
- Produced first insights: staff roles, roster size, hometown slicing.

Week 4 - Identity and Season Awareness

- Strengthened player identity to avoid duplicates.
- Introduced a minimal season concept to support longitudinal questions.
- Planned for year-over-year comparisons and jersey reuse.

Week 5 - 15 Schools + Attribute Modeling

- Extracted 2025 rosters for 15 universities.
- Modeled jersey/height/height inches/weight/BMI as nodes.
- Height normalization improved significantly.
- BMI computed and linked for physique-related analysis.

Week 6 - Multi-Year Plan

- Planned 2024 expansion and multi-season queries.
- Planned position normalization and richer analysis queries.
- Prepared query library for evaluation and QA.

Week 7 - 20 Schools + Parser Robustness

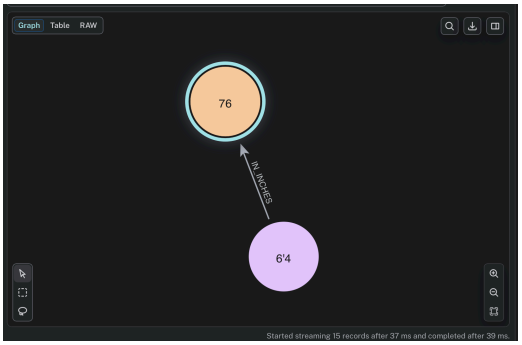
- Completed 2025 roster import for 20 universities.
- Documented parsing challenges across schools.
- Improved normalization for height and position inconsistencies.
- Added error handling so bad rows do not crash pipeline.

8. Data Cleaning and Normalization

8.1 Height normalization

Schools report height in inconsistent formats ("6-3", "6'3", "6 ft 3 in", unusual punctuation). DiamondKG normalizes:

- a consistent feet-inches representation
- a numeric inches representation (HeightInches)




```
n
1 (:BMI {value: 25.8})
2 (:BMI {value: 27.1})
3 (:BMI {value: 28.9})
```

```python
# Raw header -> canonical header
CANON_MAP = {
    # jersey / roster number
    "No.": "jersey",
    "#": "jersey",
    "–": "jersey",
    "Unnamed: 1": "jersey",
    "No" : "jersey",
    "Number" : "jersey",


    # names
    "Name": "name",
    "Full Name": "name",

    # year/class
    "Year": "year",
    "Academic Year": "year",
    "Yr.": "year",
    "Class" : "year",
    "Cl." : "year",

    # position
    "Position": "position",
    "Pos.": "position",
    "Pos" :"position",

    # bats/throws
    "Bats/Throws": "bats_throws",
    "B/T": "bats_throws",
```

```python
# Normalize different quote variants
s = s.replace("", "'").replace("`", "'")
# Remove double quotes and double single-quotes
s = s.replace('"', "").replace("''", "")
# Collapse multiple spaces
s = re.sub(r"\s+", " ", s)

# Case 1: "6-0" -> "6'0"
if "–" in s:
    feet, inches = s.split("–", 1)
    feet = feet.strip()
    inches = inches.strip()

    # keep only digits from each part (just in case)
    m_feet = re.search(r"\d+", feet)
    m_inch = re.search(r"\d+", inches)
    if m_feet and m_inch:
        return f"{m_feet.group()}'{m_inch.group()}"
    else:
        return s  # fallback

# Case 2: "5'11", "5' 11", "6'0", "6' 0"
m = re.search(r"(\d+)\s*'\s*(\d+)", s)
if m:
    feet, inches = m.groups()
    return f"{feet}'{inches}"

# Fallback: leave it unchanged if we don't recognize pattern
return s
```

This enables comparisons like “average height by position” and avoids brittle string matching.

8.2 Position normalization

Position strings vary widely (RHP vs Pitcher vs Right-Handed Pitcher). The pipeline:

- preserves raw position text for traceability
- maps to canonical position categories to reduce fragmentation
- supports multi-position cases by splitting combined values (e.g., IF/RHP)

8.3 Weight and BMI

Weight is cleaned to numeric form when possible. BMI is computed only when height-inches and weight are available and then stored as a BMI node. This supports group comparisons like “BMI by position” across schools.

8.4 Majors and previous school

These are optional and not present on every site. When available:

- major values connect to Major nodes
- previous school values connect to PreviousSchool nodes

Even partial coverage is useful for cross-school summaries.

9. Knowledge Graph Loading Strategy and No-Repeatability

At scale, imports must be re-runnable. DiamondKG is designed so loading the same data again does not create duplicates. This is achieved through:

- deterministic identifiers (stable keys)
- uniqueness constraints
- idempotent merge behavior

This ensures:

- schools remain unique
- players remain unique
- season snapshots remain unique
- re-running the pipeline updates missing properties instead of duplicating nodes

10. Query Evaluation and Results

This section describes how DiamondKG was evaluated through queries. First, baseline queries validate that the KG remained consistent after schema upgrades. Then, multi-season queries demonstrate new analytical power enabled by PlayerSeason.

10.1 Sample Query Outputs (Suggested).

Figure 1: Nodes in Neo4j Browser

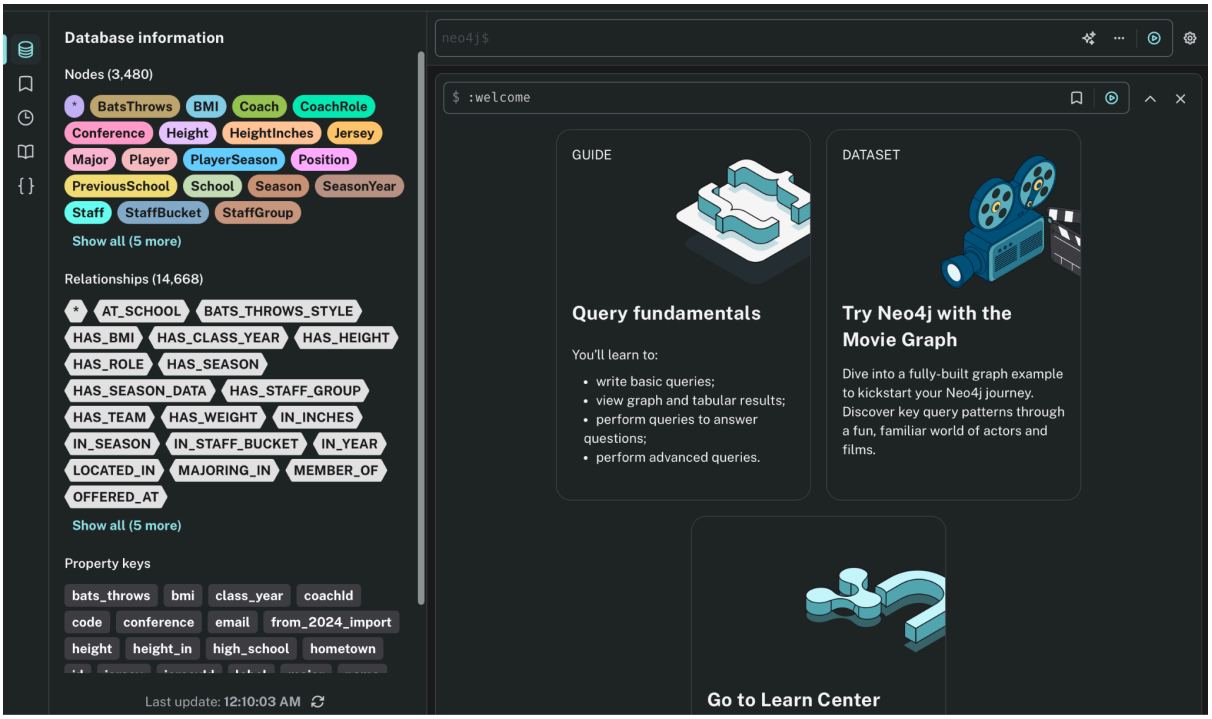


Figure 2: Top 5 Schools with Coaches Count

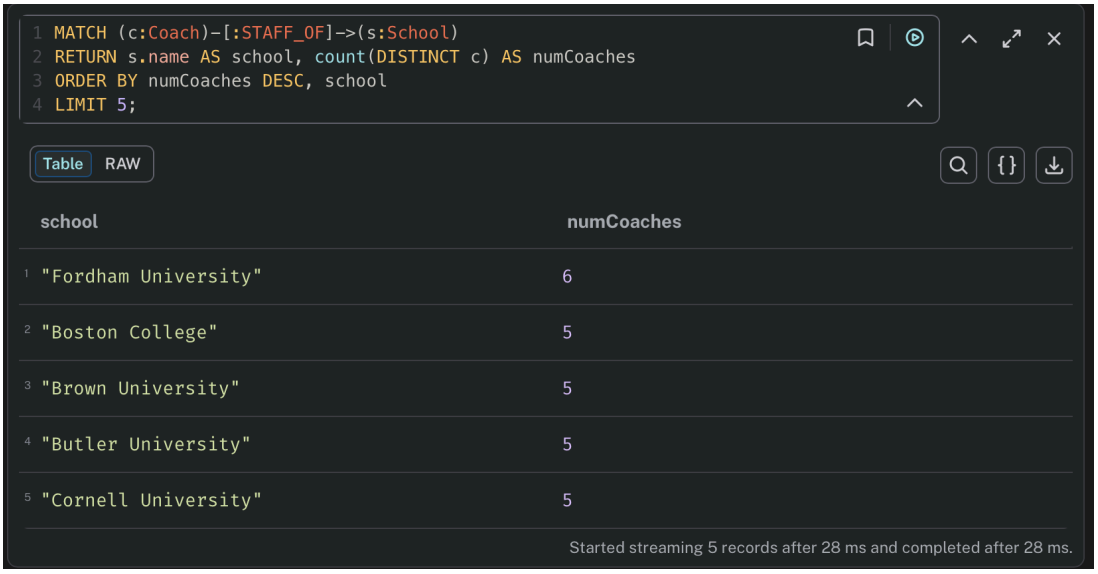


Figure 3: Players/Positions Distribution Output

```
MATCH (p:Player)-[:PLAYS_POSITION]->(pos:Position)
RETURN pos.name AS Position, count(DISTINCT p) AS Players
ORDER BY Players DESC;
```

Table RAW

Position	Players
"RHP"	348
"OF"	211
"INF"	157
"LHP"	124
"C"	107
"P"	100
"1B"	49
"IF"	36
"UTIL"	22
"3B"	7

Started streaming 18 records after 40 ms and completed after 44 ms.

Figure 4: Top Majors Output

```
MATCH (p:Player)-[:MAJORING_IN]->(m:Major)
RETURN m.name AS Major, count(DISTINCT p) AS Players
ORDER BY Players DESC
LIMIT 10;
```

Table RAW

Major	Players
"Sport Management"	9
"Undeclared"	7
"Exercise Science"	6
"Finance"	5
"Sports Management"	4
"Management"	3
"Undecided"	3
"Entrepreneurship"	2
"Exercise Physiology"	2
"Business Administration"	2

Started streaming 10 records after 37 ms and completed after 39 ms.

Figure 5: Transfer Detection Output

```
MATCH (p:Player)-[:PREVIOUS_SCHOOL]->(ps:PreviousSchool)
MATCH (p)-[:PLAYS_FOR]->(s:School)
RETURN
  s.name AS currentSchool,
  count(DISTINCT p) AS numTransferredPlayers
ORDER BY numTransferredPlayers DESC, currentSchool;
```

Table RAW

	currentSchool	numTransferredPlayers
1	"University of Georgia"	32
2	"Ball State University"	24
3	"Butler University"	24
4	"Iona University"	20
5	"Elon University"	14
6	"Monmouth University"	14
7	"Saint Joseph's University"	12
8	"Georgetown University"	11
9	"Gannon University"	7
10	"Villanova University"	3

10.2 Baseline consistency queries (what they prove)

- Position distributions remain realistic and not corrupted by normalization.
- Major distributions remain stable after adding seasons.
- Coach-to-school links remain intact after player schema changes.
- Height and BMI relationships remain correct after multi-season additions.
- School-level counts match expectations (no missing schools, no empty rosters).

10.3 Multi-season analysis (what becomes possible)

With PlayerSeason:

- returning players per school (roster stability)
- player attribute changes across years (position/weight/class year)
- transfer detection between schools
- year-by-year roster comparisons at school and conference level

11. Limitations, Risks, and Mitigations

Limitations

- HTML structure varies across schools; requires per-school adjustments.
- Majors and previous schools are missing for many programs.
- Position text can be inconsistent without careful canonicalization.
- Full transfer detection is strongest once multiple seasons are loaded widely.

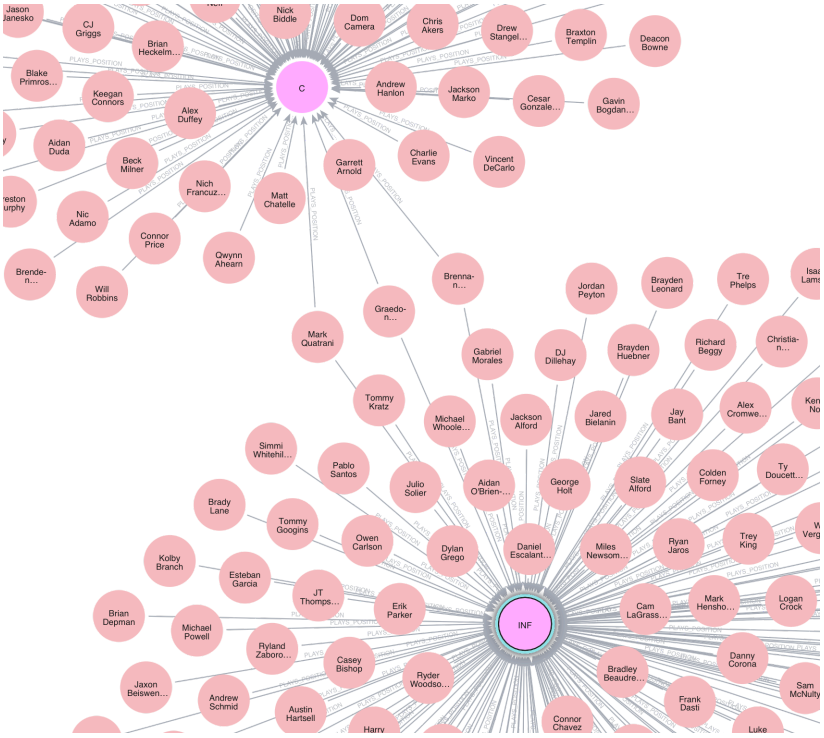
Risks and mitigations

- Website redesign breaks selectors → use provenance + QA checks + per-school config.
- Duplicate people due to name collisions → deterministic IDs + reified PlayerSeason.
- Missing values create incomplete relationships → consistent missing-value policy + audits.
- Roster updates over time → scheduled recrawls + idempotent loading.

12. Changes Implemented After Feedback

After receiving feedback from the professor, I made several targeted improvements to ensure the knowledge graph is more **consistent**, **season-aware**, and **scalable**, while also improving query quality and preventing duplication during repeated loads.

12.1 Split Position Values

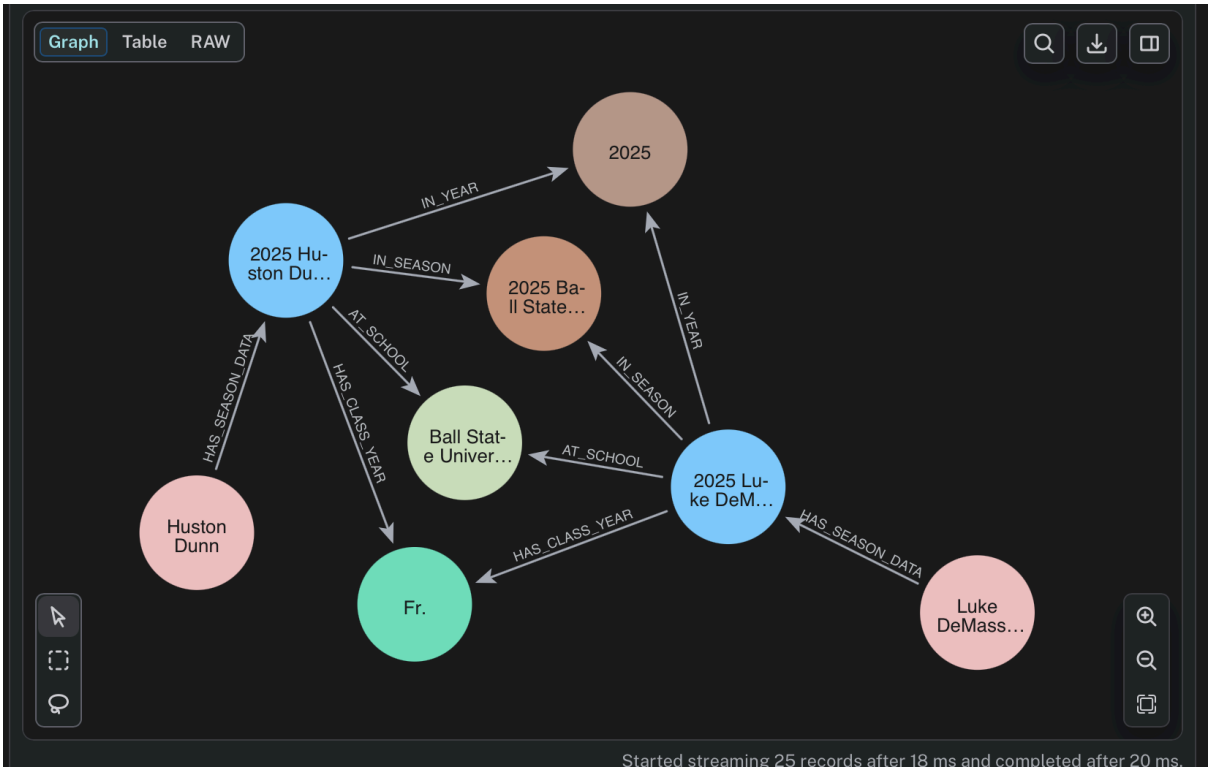


Problem: Some schools list multi-role positions like “IF/RHP” or inconsistent strings like “Pitcher”, “RHP”, “Right-Handed Pitcher”.
Change made: I updated the parsing and modeling so that:

- Composite positions are **split** and linked to multiple Position nodes when appropriate.
- Raw position strings are preserved for traceability, but the graph supports a cleaner normalized view for aggregation.

Benefit: Position-based queries (counts, distributions, comparisons across schools) are more accurate and less fragmented.

12.2 Explicit Year Handling



Problem: A single-season model cannot correctly represent changes over time (jersey reuse, class-year progression, weight changes).
Change made: I introduced explicit **Year/Season modeling**, so 2024 and 2025 can coexist without overwriting each other.

- Added a **Year node** (or Year field) and linked season-scoped records to it.

Benefit: Enables year-by-year comparisons and prevents mixing 2024 and 2025 data.

12.3 No-Repeatability (Duplicate Prevention) — Two Layers of Protection

Problem: Re-running ETL loads can create duplicates in Neo4j if identity rules are weak.
Change made: I enforced **no-repeatability** using two complementary mechanisms:

1. **Uniqueness constraints / deterministic IDs**
    - Schools are unique (e.g., by school name or schoolId).
    - Season-scoped entities use stable keys (e.g., PlayerSeason key based on player + school + year).
2. **Idempotent loading logic (upserts)**

- Loads are designed so rerunning the same import updates existing nodes/relationships rather than creating new ones.

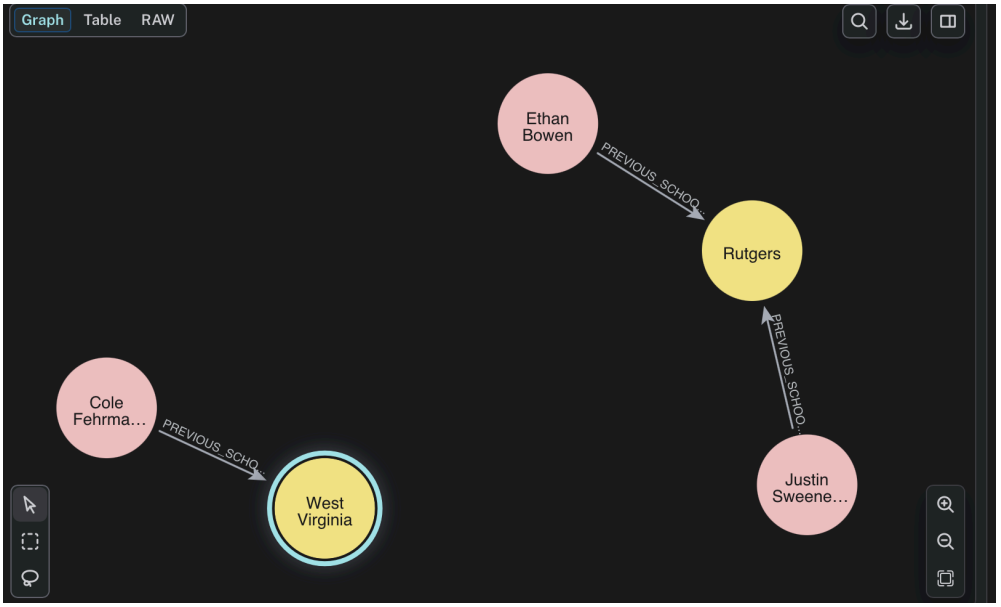
Benefit: The KG remains stable across re-crawls and re-imports (counts do not inflate over time).

12.4 Major as a Node

Problem: Major values are useful for analysis but become messy if stored only as raw strings.
Change made: I modeled **Major** as a node and linked players (or PlayerSeason snapshots) to it.

Benefit: Supports clean aggregation queries such as "Top majors across the dataset" and "Major distribution by school/season".

12.5 Previous School as a Node



Problem: Previous school values vary in format (JUCO, HS, colleges, mixed text).
Change made: I added a **PreviousSchool node** and connected it to the appropriate player season entries when present.

Benefit: Enables analysis like "most common feeder schools", and supports transfer-related interpretation.

12.6 Reification in Neo4j (PlayerSeason)



Problem: The fact "a player is on a school roster in a specific year" carries multiple attributes (jersey, class year, position, weight) and cannot be cleanly represented as a single edge without losing detail.
Change made: I implemented **reification** using a **PlayerSeason** node representing:

> "This player at this school in this season."

Season-varying attributes (jersey, class year, position, weight, major, previous school) are stored on or linked from PlayerSeason instead of overwriting Player.

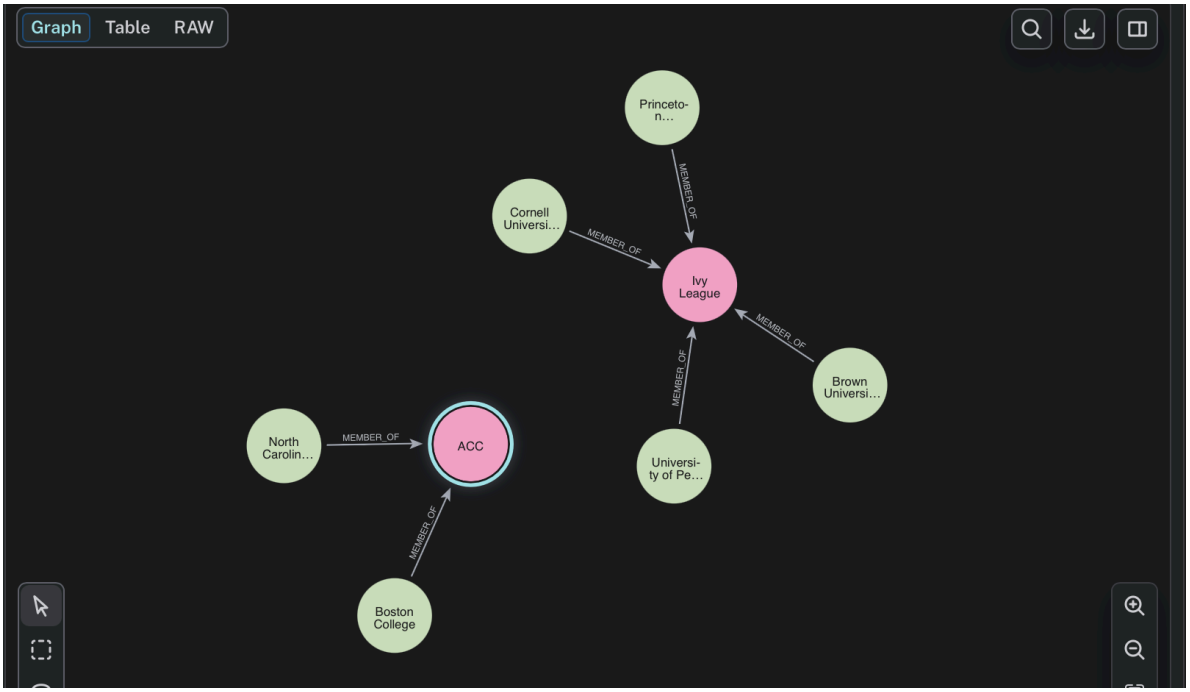
Benefit: This makes the graph season-aware, prevents conflicts across years, and enables longitudinal analysis.

12.7 Transfer Analysis

Problem: Transfer detection is not reliable without multi-season structure.
Change made: After introducing Year + PlayerSeason, I enabled **transfer detection** by identifying the same Player connected to different schools across seasons (e.g., 2024 at School A, 2025 at School B).

Benefit: Supports transfer candidate queries and school-level summaries of gained/lost players.

12.8 Conference and Team Modeling (For Higher-Level Aggregation)



Problem: School-to-school comparisons are helpful, but conference-level comparisons require Team/Conference structure.
Change made: I added:

- **Team** nodes linked to School
- **Conference** nodes linked to Team (and optionally season-aware membership)

Benefit: Enables conference-level summaries such as "average BMI by conference" or "roster size by conference".

13. Conclusion

13. Conclusion

This project successfully built an end-to-end Neo4j knowledge graph (DiamondKG) for U.S. college baseball rosters and staff using official athletics websites as the data source. Over the semester, the work evolved from an initial proposal and minimal ontology into a scalable pipeline (discovery → scraping → cleaning/normalization → publishing → Neo4j loading → query evaluation) that supports cross-school analysis and is designed to be re-runnable without creating duplicates.

After my meeting, I implemented the suggested improvements that significantly strengthened the graph's reliability and analytical value. In particular, I (1) improved position handling by splitting composite positions and reducing fragmentation, (2) added explicit year/season handling, (3) introduced reification through season-scoped player snapshots (PlayerSeason) so year-to-year changes are modeled correctly, (4) added Major

and PreviousSchool as nodes to support richer academic and mobility queries, (5) enabled transfer-style analysis across seasons, and (6) structured staff into clearer buckets (Coaches vs Staff vs Support Staff) so coach counts and role-based comparisons remain accurate. I also ensured no-repeatability through deterministic identifiers and idempotent loading so that re-importing data does not inflate counts.

To make the work easy to evaluate, I included a large number of screenshots throughout the report showing the schema, example subgraphs, and query outputs. These visuals demonstrate that the KG is not only structurally consistent, but also capable of answering meaningful questions such as top majors, roster distributions by position, roster stability across years, class-year progression (e.g., Junior → Senior), and transfer candidates.

Overall, DiamondKG meets the project goals of creating a clean, traceable, and extensible knowledge graph. As future work, I plan to complete full multi-season coverage (2024 and 2025 for all schools), finalize conference/team membership by season, and expand quality checks and analytics (conference-level comparisons, transfer flows, and longitudinal roster change summaries). Thank you for reviewing my project and for the feedback that helped improve it.